

ARTIFICIAL INTELLIGENCE AND DEEP LEARNING

LAB ASSIGNMENT – 07

NAME – SUFIYA AFREEN

ENROLLMENT NO – 2403A51067

YEAR/SEM – 3/1

DATE – 3-9-26

BRANCH/BATCH – CSE/03

Lab/Exp 7: Building a CNN for Image Recognition.

Kaggle Dataset Link: [https://www.kaggle.com/datasets/ Fashionmnist-dataset](https://www.kaggle.com/datasets/Fashionmnist-dataset)

Tasks:

1. Load and preprocess the FashionMNIST dataset.
 2. Implement the required PyTorch model.
 3. Train using suitable hyperparameters.
 4. Evaluate using appropriate metrics.
 5. Visualize results (loss/accuracy/features as applicable).
 6. Summarize observations and conclusions.
-

```
[1]
✓ 24s
# Lab/Exp 7: Building a CNN for Image Recognition
# Dataset: FashionMNIST
# Framework: PyTorch

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms

from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
import numpy as np

from sklearn.metrics import classification_report, confusion_matrix
import seaborn as sns
```

```
[2]
✓ 0s
# Check whether GPU is available

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

print("Device:", device)

if torch.cuda.is_available():
    print("GPU:", torch.cuda.get_device_name(0))

*** Device: cpu
```

```
# Transform:
# Convert images to tensors
# Normalize pixel values

transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5,), (0.5,))
])

# Training dataset
train_dataset = torchvision.datasets.FashionMNIST(
    root="./data",
    train=True,
    transform=transform,
    download=True
)

# Testing dataset
test_dataset = torchvision.datasets.FashionMNIST(
    root="./data",
    train=False,
    transform=transform,
    download=True
)

# Data loaders
train_loader = DataLoader(
    train_dataset,
    batch_size=64,
    shuffle=True
)

test_loader = DataLoader(
    test_dataset,
    batch_size=64,
    shuffle=False
)

print("Training samples:", len(train_dataset))
print("Testing samples:", len(test_dataset))

*** 100%|██████████| 26.4M/26.4M [00:01<00:00, 17.2MB/s]
100%|██████████| 29.5k/29.5k [00:00<00:00, 270kB/s]
100%|██████████| 4.42M/4.42M [00:00<00:00, 5.01MB/s]
100%|██████████| 5.15k/5.15k [00:00<00:00, 11.8MB/s]Training samples: 60000
Testing samples: 10000
```

```
[4]
✓ On
classes = [
    "T-shirt/top",
    "Trouser",
    "Pullover",
    "Dress",
    "Coat",
    "Sandal",
    "Shirt",
    "Sneaker",
    "Bag",
    "Ankle boot"
]

# Get one batch
images, labels = next(iter(train_loader))

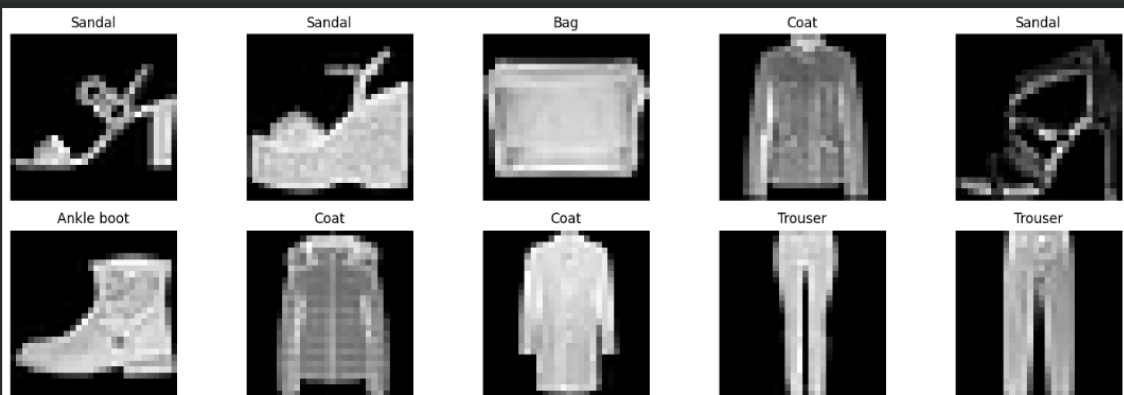
# Display 10 images
plt.figure(figsize=(15, 5))

for i in range(10):
    plt.subplot(2, 5, i + 1)

    # Unnormalize
    image = images[i] * 0.5 + 0.5

    plt.imshow(image.squeeze(), cmap="gray")
    plt.title(classes[labels[i]])
    plt.axis("off")

plt.tight_layout()
plt.show()
```



```

class CNN(nn.Module):

    def __init__(self):
        super(CNN, self).__init__()

        # Convolution Layer 1
        self.conv1 = nn.Conv2d(
            in_channels=1,
            out_channels=32,
            kernel_size=3,
            padding=1
        )

        # Convolution Layer 2
        self.conv2 = nn.Conv2d(
            in_channels=32,
            out_channels=64,
            kernel_size=3,
            padding=1
        )

        # Pooling
        self.pool = nn.MaxPool2d(kernel_size=2, stride=2)

        # Dropout
        self.dropout = nn.Dropout(0.25)

        # Fully connected layers
        self.fc1 = nn.Linear(64 * 7 * 7, 128)
        self.fc2 = nn.Linear(128, 10)

    def forward(self, x):

        # Conv Block 1
        x = torch.relu(self.conv1(x))
        x = self.pool(x)

        # Conv Block 2
        x = torch.relu(self.conv2(x))
        x = self.pool(x)

        # Flatten
        x = x.view(x.size(0), -1)

        # Fully Connected
        x = torch.relu(self.fc1(x))

        # Dropout
        x = self.dropout(x)

        # Output
        x = self.fc2(x)

        return x

```

```

... CNN(
  (conv1): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (conv2): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
  (pool): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  (dropout): Dropout(p=0.25, inplace=False)
  (fc1): Linear(in_features=3136, out_features=128, bias=True)
  (fc2): Linear(in_features=128, out_features=10, bias=True)
)

```

Input Image ↓ $28 \times 28 \times 1$ ↓ Conv2D (32 filters) ↓ ReLU ↓ Max Pooling ↓ $14 \times 14 \times 32$ ↓ Conv2D (64 filters) ↓ ReLU ↓ Max Pooling ↓ $7 \times 7 \times 64$ ↓ Flatten ↓ Fully Connected (128) ↓ Dropout ↓ Fully Connected (10) ↓ Output Classes

```

train_accuracies = []

for epoch in range(epochs):

    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:

        # Move data to GPU/CPU
        images = images.to(device)
        labels = labels.to(device)

        # Clear gradients
        optimizer.zero_grad()

        # Forward pass
        outputs = model(images)

        # Calculate loss
        loss = criterion(outputs, labels)

        # Backpropagation
        loss.backward()

        # Update weights
        optimizer.step()

        # Statistics
        running_loss += loss.item()

    _, predicted = torch.max(outputs.data, 1)

    total += labels.size(0)
    correct += (predicted == labels).sum().item()

    epoch_loss = running_loss / len(train_loader)
    epoch_accuracy = 100 * correct / total

    train_losses.append(epoch_loss)
    train_accuracies.append(epoch_accuracy)

    print(
        f"Epoch [{epoch+1}/{epochs}] "
        f"Loss: {epoch_loss:.4f} "
        f"Accuracy: {epoch_accuracy:.2f}%"
    )

```

```

*** Epoch [1/10] Loss: 0.4758 Accuracy: 82.75%
Epoch [2/10] Loss: 0.3052 Accuracy: 88.93%
Epoch [3/10] Loss: 0.2562 Accuracy: 90.54%
Epoch [4/10] Loss: 0.2258 Accuracy: 91.71%
Epoch [5/10] Loss: 0.2045 Accuracy: 92.46%
Epoch [6/10] Loss: 0.1839 Accuracy: 93.22%

```

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, epochs + 1),
    train_losses,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Training Loss")
plt.title("Training Loss vs Epoch")
plt.grid()

plt.show()
```

```
plt.figure(figsize=(8, 5))

plt.plot(
    range(1, epochs + 1),
    train_accuracies,
    marker="o"
)

plt.xlabel("Epoch")
plt.ylabel("Training Accuracy (%)")
plt.title("Training Accuracy vs Epoch")
plt.grid()

plt.show()
```

```
model.eval()

correct = 0
total = 0

all_predictions = []
all_labels = []

with torch.no_grad():
    for images, labels in test_loader:

        images = images.to(device)
        labels = labels.to(device)

        outputs = model(images)

        _, predicted = torch.max(outputs.data, 1)

        total += labels.size(0)
        correct += (predicted == labels).sum().item()

    all_predictions.extend(predicted.cpu().numpy())
    all_labels.extend(labels.cpu().numpy())
```

----- THANKYOU -----